Chapter 78
Two and Higher Dimensional Pattern Matching in Optimal Expected Time*

Juha Kärkkäinen† Esko Ukkonen‡

Abstract

Algorithms with optimal expected running time are presented for searching the occurrences of a two-dimensional $m \times m$ pattern P in a two-dimensional $n \times n$ text T over an alphabet of size c. The algorithms are based on placing in the text a static grid of test points, determined only by n, m and c (not dynamically by earlier test results). Using test strings read from the test points the algorithms eliminate as many potential occurrences of P as possible. The remaining potential occurrences are separately checked for actual occurrences. By suitably choosing the test point set, we obtain algorithms with expected running time $O(\frac{n^2}{m^2}\log_c m^2)$ which is optimal by a lower bound result by Yao. The method is generalized for the k mismatches problem. The resulting algorithm has expected running time $O(\frac{n^2}{m^2}(k+1)\log_c m^2)$, provided that $k \leq m^2/(4\lceil\log_c m^2\rceil)$. All algorithms need preprocessing of P which takes time and space $O(m^2 \log_c m^2)$ or $O(m^2)$. The text processing can be done on-line, using a rather small window. The algorithms easily generalize to d-dimensional matching for any d.

1 Introduction

The classical *pattern matching problem* for strings is to find all (exact or approximate) occurrences of a *pattern* P in a *text* T. In one-dimensional case P and T are strings over some alphabet Σ of size c. In d-dimensional case P and T are d-dimensional arrays over Σ. The applications are numerous.

The exact one-dimensional pattern matching is well-understood. For two and higher dimensional versions several algorithms have been proposed, too, mainly aiming at good worst-case running time; see e.g. [6,5,14,21,2]. Recently, the two-dimensional problem has been solved in linear worst-case time with algorithms that are alphabet-independent in the same sense as the one-dimensional Knuth-Morris-Pratt algorithm [3,11]. A space-economical solution is given in [9].

In the applications of one-dimensional matching the Boyer-Moore algorithm [7] whose expected running time is "sublinear" is clearly the most successful; see e.g. [13]. Also in higher dimensions one should look at algorithms with good expected running time, not only the worst case. The recent algorithm by Baeza-Yates and Regnier [4] is a significant step in this direction; their algorithm finds the occurrences of a $m \times m$ pattern in a $n \times n$ text in expected time $O(n^2/m)$.

In this paper we give better algorithms that find exact occurrences of a $m \times m$ pattern in a $n \times n$ text in expected time $O(\frac{n^2}{m^2}\log_c m^2)$; we use the Bernoulli model of random strings over Σ. The algorithms are not dimensionality specific. For d-dimensional P of size m^d and T of size n^d they run in expected time $O(\frac{n^d}{m^d}\log_c m^d)$.

In [20], A. Yao confirmed a conjecture of Knuth [16] by proving a lower bound of $O(\frac{n}{m}\log_c m)$ for one-dimensional case (when $n \geq 2m$). Yao's argument generalizes for two and higher dimensional matching and gives a general lower bound $O(\frac{|T|}{|P|}\log_c |P|)$. Hence our algorithms are the first algorithms for multidimensional matching with optimal expected running time. They are conceptually simple and have low overhead.

We also generalize our algorithm for approximate pattern matching. We consider the two-dimensional k *mismatches problem* (find all occurrences of P in T with $\leq k$ mismatching symbols) and give an algorithm with expected running time $O(\frac{n^2}{m^2}(k+1)\log_c m^2)$. The algorithm requires that $k \leq m^2/(4\lceil\log_c m^2\rceil)$. Under this condition the expected running time is at most linear in $|T|$; previously Chang and Lawler [8] (see also [19]) have given an algorithm with similar properties for the k differences problem of one-dimensional approximate matching. The two-dimensional k mismatches problem has also been studied in [17].

The naive algorithm for pattern matching and its improvements such as the Knuth-Morris-Pratt algorithm [16] are based on a view of the problem that could be called *positive*: the algorithm tries to prove at each text location that there is an occurrence of P at that location.

*Work supported by the Academy of Finland.

†tpkarkka@cs.Helsinki.FI, Department of Computer Science, P.O. Box 26 (Teollisuuskatu 23), SF-00014 University of Helsinki, Finland.

‡ukkonen@cs.Helsinki.FI, Department of Computer Science, P.O. Box 26 (Teollisuuskatu 23), SF-00014 University of Helsinki, Finland.

In applications T often contains relatively few occurrences of P. Hence the *negative* view can fit the situation better than the positive one. Then the algorithm tries to prove for each text location that there can *not* be an occurrence of P at that location. This forms the *elimination phase* of the algorithm. Whenever the elimination proof fails the algorithm has found a potential occurrence of P that needs further consideration. A separate *checking phase* then examines whether or not a potential occurrence really contains P. This leads to fast algorithms because the elimination phase needs on the average to examine only a small fraction of T.

The well-known Boyer-Moore algorithm [7] has this flavour. In this paper we develop algorithms based on the negative view further for two and higher dimensional pattern matching.

We explain the idea of our algorithms using two dimensional P and T. Let

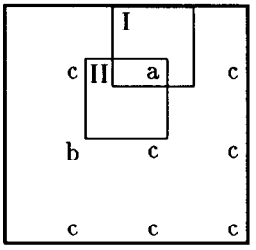
P:

b	b	a
b	b	b
b	a	b

T:

a	a	a	a	b	b	a	b	b
b	b	b	b	b	b	b	b	b
a	b	c	c	b	a	b	a	c
b	a	b	a	b	a	b	a	b
a	a	c	b	b	b	a	a	a
b	c	b	b	b	c	b	b	c
a	c	c	b	c	c	a	c	c
c	c	c	b	b	b	a	a	a
c	c	c	c	a	c	b	b	c

We have to find all (I, J) such that $T[I+k-1, J+h-1] = P[k, h]$ for $1 \leq k, h \leq 3$. If we are lucky we can eliminate all possible occurrences of P in T by examining only the entries $T[i, j]$ where $i = 3, 6, 9$, $j = 3, 6, 9$, because any possible occurrence of P in T must overlap at least one of these entries of T. The test entries in T are as follows



As the first entry $T[3, 3]$ equals c, and c does not occur in P at all, no occurrence of P in T can overlap $T[3, 3]$. This proves that there can not be an occurrence of P inside subarray $T[1 : 5, 1 : 5]$. Test entry $T[3, 6] = a$ eliminates all other occurrences of P that overlap $T[3, 6]$ except those that have overlapping symbol a. The upper left corners of these potential occurrences of P are $T[1, 5]$ (=I) and $T[3, 4]$ (=II), because $P[1, 3] = P[3, 2] = a$. The checking phase then finds that there is an occurrence of P at $T[1, 5]$ but not at $T[3, 4]$.

The fourth test entry $T[6, 3] = b$ is least useful in the elimination because b has 7 occurrences in P. Hence we are left with 7 potential occurrences of P that overlap $T[6, 3]$. Here we note that taking $T[6, 2] = c$ as an additional test entry would eliminate all possible occurrences of P that overlap $T[6, 2]$ and $T[6, 3]$ because substring cb does not occur in P.

As increasing the number of test entries obviously leads to more extensive elimination, we determine q such that when the elimination is based on q entries of T, the expected running time of elimination and (naive) checking gets minimum. It turns out that minimum is given by $q \approx \log_{|\Sigma|} |P|$; this simply has the effect that we are using a superalphabet of size $\Theta(|P|)$.

The test entries will be distributed over T such that every potential occurrence of P in T covers at least q entries. We use *static distribution* of test entries; this is a major difference to virtually all other pattern matching algorithms proposed in the literature. The grid of test entries is determined using only the size (and shape) of P, T and the underlying alphabet. The elimination phase becomes *oblivious*: the results of earlier elimination tests do not cause dynamic changes on the later tests. This property helps in writing fast code for the algorithms, for both sequential and parallel environments. —After completing this paper we found out that the idea of using superalphabet of size $\Theta(|P|)$ in two-dimensional matching has very recently appeared or will appear in [10,15,18]. Crochemore and Rytter [10] sketch an algorithm which comes close to one of our algorithms, while the algorithms in [15,18] differ in that they determine the test entries dynamically.

The above idea is formulated in Section 2 as a generic pattern matching algorithm in a very general setting. The elimination of potential occurrences of P is based on reading test strings of length q from T. The strings are picked up according to a fixed sampling scheme. In Section 3 we give an example scheme such that each test entry belongs to only one test string. To get minimal total number of test entries, we use test strings whose shape has smallest possible diameter. In Section 4 we give a bit more efficient scheme in which a test entry belongs to several test templates. We could distribute test entries to an evenly spaced grid over T. As doing pattern matching over the grid is as complicated as the original problem — the size of the text has only been reduced to $\frac{|T|}{|P|}q$ — we use test templates that belong to one-dimensional subset of T. Then the elimination phase can be implemented efficiently using one-dimensional multipattern matching. Section 5 reformulates the lower bound of Yao [20] for d-dimensional pattern matching. In Section 6 we generalize the algorithm for the k mismatches problem. The elimination is based on $k + p$ non-overlapping test strings; here p is a parameter used to optimize the performance of the method.

2 General Algorithm

To keep the technical exposition simple we restrict ourselves to the two-dimensional case with square patterns and texts. The generalization of all our results to the d-dimensional pattern matching with rectangular patterns and texts is straightforward.

Let a two-dimensional $m \times m$ array $P = (p_{ij})$, $1 \leq i, j \leq m$, be the *pattern* and a 2-dimensional $n \times n$ array $T = (t_{ij})$, $1 \leq i, j \leq m$, be the *text* over some alphabet Σ. Let c denote the size of alphabet Σ.

We let integer q denote the *sample size*. Selecting a suitable q will minimize the expected running time of our algorithms. A sequence $Q = ((i_1, j_1), (i_2, j_2), \ldots, (i_{q-1}, j_{q-1}))$ of $q-1$ *disjoint* pairs of indexes is called a (two-dimensional) *template* of size q. A template may not contain the pair $(0,0)$ which is always the *origin* of the template. A template Q and a pair of indexes (i, j) define a *sample* $Q(i,j) = ((i,j), (i+i_1, j+j_1), \ldots, (i+i_{q-1}, j+j_{q-1}))$. Here Q gives the shape and (i,j) the location of the origin of sample $Q(i,j)$.

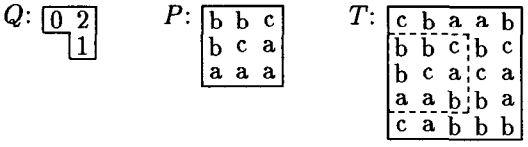
Pattern P is said to *contain* sample $Q(i,j)$ if the pairs listed in $Q(i,j)$ refer to the entries of P. Such sample $Q(i,j)$ determines a *test string* $s(P, Q(i,j))$ in P as $s(P, Q(i,j)) = p_{i,j}p_{i+i_1,j+j_1} \cdots p_{i+i_{q-1},j+j_{q-1}}$.

A *sampling scheme* $\mathcal{Q}$ for a template Q is a pair $(\mathcal{Q}_P, \mathcal{Q}_T)$ where *pattern sampling scheme* $\mathcal{Q}_P$ and *text sampling scheme* $\mathcal{Q}_T$ are subsets of all possible Q-shaped samples of P and T, respectively, that is

$$\begin{aligned} \mathcal{Q}_P &\subseteq \{Q(i,j) \mid P \text{ contains } Q(i,j)\} \text{ and} \\ \mathcal{Q}_T &\subseteq \{Q(i,j) \mid T \text{ contains } Q(i,j)\}. \end{aligned}$$

Let R be a potential occurrence of P in T (i.e., an $m \times m$ subarray of T) with upper left corner at (i_R, j_R). A *witness* of R is pair $(Q(i_P, j_P), Q(i_T, j_T)) \in \mathcal{Q}_P \times \mathcal{Q}_T$ such that $i_T - i_P + 1 = i_R$ and $j_T - j_P + 1 = j_R$, i.e., $Q(i_T, j_T)$ is in R in the same position as $Q(i_P, j_P)$ is in P. Witness $(Q(i_P, j_P), Q(i_T, j_T))$ is positive if $s(P, Q(i_P, j_P)) = s(T, Q(i_T, j_T))$; otherwise the witness is negative. A potentential occurrence with negative witness can not be a real occurrence of P in T.

Example. Let



Template $Q = ((1,1),(0,1))$ is shown here with the origin. Let R be a potential occurrence of P in T with upper left corner at $(2,1)$ (marked with dashed boundaries above). If now $Q(2,2) \in \mathcal{Q}_P$ and $Q(3,2) \in \mathcal{Q}_T$, then (Q(2,2),Q(3,2)) is a negative witness of R as $s(P, Q(2,2)) = caa \neq cba = s(T, Q(3,2))$. This proves that R is not an occurrence of P. All other possible witnesses of R would be positive. □

We are now ready to give a general description of our algorithms. The idea is to eliminate potential occurrences with at least one negative witness and check the remaining potential occurrences using, e.g., naive checking. An efficient way to do this is to select a sampling scheme such that every potential occurrence has exactly one witness in it. Then the potential occurrences worth checking can be determined by finding all positive witnesses. Uninteresting potential occurrences are implicitly eliminated by ignoring negative witnesses.

ALGORITHM 2.1. (GENERIC)

1. [q, Q and $\mathcal{Q}$] Select suitable sample size q, template Q and sampling scheme $\mathcal{Q} = (\mathcal{Q}_P, \mathcal{Q}_T)$ for given pattern P and text T over alphabet Σ of size c. Scheme $\mathcal{Q}$ must provide exactly one witness for each potential occurrence of P in T.

2. [Preprocessing of P] For each pattern sample $Q(i,j) \in \mathcal{Q}_P$, evaluate the test string $s(P, Q(i,j))$. Construct a suitable data structure to quickly find all pattern samples with a given test string.

3. [Elimination phase] For each text sample $Q(i_T, j_T) \in \mathcal{Q}_T$, scan T to obtain $s(T, Q(i_T, j_T))$ and find, using the data structure constructed in the preprocessing step, all pattern samples $Q(i_P, j_P)$ such that $s(P, Q(i_P, j_P)) = s(T, Q(i_T, i_T))$. For each matching pair $(Q(i_P, j_P), Q(i_T, j_T))$, execute the checking phase for the potential occurence R with upper left-hand corner at $(i_R, j_R) = (i_T - i_P + 1, j_T - j_P + 1)$.

4. [Checking phase] Given potential occurrence R with upper left-hand corner at (i_R, j_R), check whether or not there really is an occurrence, that is, test whether or not $t_{i_R+I-1, j_R+J-1} = p_{I,J}$ for $1 \leq I, J \leq m$. □

Selecting different sampling schemes in the above generic algorithm gives different concrete algorithms. The simplest alternative is developed in Section 3; we use templates with small diameter to maximize the number of possible pattern samples. Utilizing overlaps of text samples leads to an algorithm with faster pattern preprocessing but more sophisticated (not necessarily slower) elimination phase. Such methods are described and analysed in Section 4.

The sampling schemes of both of these algorithms are designed to minimize the number of text positions inspected during elimination as the text is typically much larger than the pattern. For the same reason our algorithms preprocess the pattern samples. In an

application, where multiple patterns are to be searched in the same static text, it might be useful to reverse the handling of text and patterns. However, we will not follow this direction further in this paper.

3 Square templates

The requirement of one witness per potential occurrence means that the number of different witnesses in Q, $|\mathcal{Q}_P||\mathcal{Q}_T|$, approximately equals the number of potential occurrences of P in T.* This implies that one can minimize the number of text samples by maximizing the number of pattern samples. This is the main idea of the algorithm presented in this section. The idea is realized by using what we call *square templates.*

DEFINITION 3.1. *Template* $Q = ((i_1, j_1), (i_2, j_2), \ldots, (i_{q-1}, j_{q-1}))$ *of size* q *is a square template iff* $0 \leq i_k, j_k \leq \lceil\sqrt{q}\rceil - 1$, *for* $1 \leq k \leq q-1$.

A square sample (sample whose shape is defined by a square template) of size q is contained in a $\lceil\sqrt{q}\rceil \times \lceil\sqrt{q}\rceil$ subarray whose upper left-hand corner is the origin of the sample. It is easy to see that the square shape maximizes the number of possible pattern samples.† Optimal sample size q will be determined in the analysis of the algorithm.

Given a square template Q, pattern sampling scheme $\mathcal{Q}_P$ now contains all possible Q-shaped pattern samples, that is

$$\mathcal{Q}_P = \{Q(i,j) \mid 1 \leq i, j \leq m - \lceil\sqrt{q}\rceil + 1\}.$$

To get one witness per potential occurrence the text samples must be placed on every $(m - \lceil\sqrt{q}\rceil + 1)$th row and column. This gives

$$\mathcal{Q}_T = \{Q(i(m - \lceil\sqrt{q}\rceil + 1), j(m - \lceil\sqrt{q}\rceil + 1)) \mid 1 \leq i, j \leq \left\lfloor \frac{n - \lceil\sqrt{q}\rceil + 1}{m - \lceil\sqrt{q}\rceil + 1} \right\rfloor\}.$$

Now each text sample is a component of the witness of $(m - \lceil\sqrt{q}\rceil + 1)^2$ different potential occurrences.

After minimizing the number of text samples we next focus on the processing time of individual text samples. For each text sample, we need to find all matching pattern samples. To speed up this, we preprocess the pattern to get a data structure which we call *test dictionary.*

Let w be a string of length q over alphabet Σ. Let $A(w)$ be the set of origins of pattern samples whose test string is w. That is,

$$A(w) = \{(i,j) \mid w = s(P, Q(i,j)),\ Q(i,j) \in \mathcal{Q}_P\}.$$

$A(w)$ is called the set of *addresses* of w. The test dictionary $D(P, \mathcal{Q}_P)$ for pattern P and sampling scheme $\mathcal{Q}_P$ of P is a data structure supporting queries that for a given string $w \in \Sigma^q$ yield the set of addresses of w.

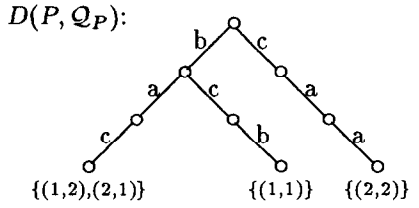
There are two obvious implementations of test dictionary $D(P, \mathcal{Q}_P)$. First, one can use a trie representing the test strings, with the addresses $A(w)$ associated with the leaf that corresponds to string w. The test strings and their addresses can be constructed by scanning P for each pattern sample. There are $\leq m^2$ samples, hence the total length of the test strings is $\leq m^2 q$, and the trie representing them can be built in time $O(m^2 qc)$ or $O(m^2 q \log c)$, depending on whether direct indexing or balanced search trees are used for implementing the branches of the trie. The queries for $A(w)$ can be answered in time $O(q)$ or $O(q \log c)$, respectively.

The second possibility to implement $D(P, \mathcal{Q}_P)$ is to convert the test strings into (c-ary) integers, and use these integers as indices to an array of size c^q representing the addresses $A(w)$ for each w. Obviously the array can be initialized and constructed in time $O(c^q + m^2 q)$, and the queries can be answered in time $O(q)$.

Example. Let Q, P, T and R be as in the example in Section 2. Then Q is a square template and, using the sampling scheme of this section, we have

$$\mathcal{Q}_P = \{Q(1,1), Q(1,2), Q(2,1), Q(2,2)\}$$
$$\mathcal{Q}_T = \{Q(2,2), Q(2,4), Q(4,2), Q(4,4)\}.$$

The witness of R in $\mathcal{Q} = (\mathcal{Q}_P, \mathcal{Q}_T)$ is $(Q(1,2), Q(2,2))$. The addresses of strings $w \in \Sigma^3$ are now $A(bac) = \{(1,2),(2,1)\}$, $A(bcb) = \{(1,1)\}$, $A(caa) = \{(2,2)\}$, and $A(w) = \emptyset$ for all other w. The trie implementation of $D(P, \mathcal{Q}_P)$ would be



3.1 Analysis.

We determine q such that the expected processing time of T (i.e., elimination + checking) is minimized under the assumption that T is a random text in the uniform Bernoulli model (each symbol of Σ can occur in a given position of T with probability $\frac{1}{c}$ independently of the

*The equality is only approximate because the potential occurrences corresponding to some witnesses near the text boundaries may not stay entirely inside the text.

†For some q, we could fit the template in a $\lceil\sqrt{q}\rceil \times \lfloor\sqrt{q}\rfloor$ subarray. For simplicity, we ignore this possibility.

content of other positions). The time is minimized by minimizing the expected time for the elimination and checking of a fixed potential occurrence R of P in T.

Let $Q(i_T, j_T) \in \mathcal{Q}_T$ be the text sample contained in R. The elimination phase for R consists of evaluating string $w = s(T, Q(i_T, j_T))$ and finding $A(w)$ from $D(P, \mathcal{Q}_P)$. This takes time $O(q)$, i.e., is proportional to the number q of entries of T that have to be examined to get w. As $Q(i_T, j_T)$ is contained in $(m - \lceil\sqrt{q}\rceil + 1)^2$ potential occurrences of P, the elimination time amortized for R is proportional to $q/(m - \lceil\sqrt{q}\rceil + 1)^2$.

The analysis of the checking phase is based on the naive checking that compares the corresponding entries of P and R until the first mismatch or an entire occurrence of P is found. The expected number of comparisons is $< \alpha = \frac{c}{c-1}$. However, the checking is activated only if the witness of R is positive. The probability of this event is $\frac{1}{c^q}$, and the expected number of comparisons is therefore $< \frac{\alpha}{c^q}$.

Now we choose q such that the total expected number of comparisons allocated for R,

$$\frac{q}{(m - \lceil\sqrt{q}\rceil + 1)^2} + \frac{\alpha}{c^q}, \tag{3.1}$$

gets minimimum. Putting the derivative of (3.1) (the ceiling function omitted) equal to zero leads to the equation

$$q = \log_c \frac{m^3}{m+1} + \log_c \frac{c \ln c}{c-1} - \log_c \frac{m}{m - \sqrt{q} + 1}$$

for the optimal (real) value of q. This further implies

$$\log_c m^2 - 1 < q < \log_c m^2 + \frac{1}{2},$$

when $m \geq 2$, $2 \leq c \leq \left(\frac{m^3}{m+1}\right)^2$, and $q < 1$ when $c > \left(\frac{m^3}{m+1}\right)^2$. Based on this we choose $q = \lceil\log_c m^2\rceil$ (when $m = 1$, define $q = 1$). Substituting this into (3.1) and noting that the potential occurrence R can be selected in $\leq n^2$ different ways gives the following bound for the expected number of symbols examined during the elimination and checking:

$$\begin{aligned} & n^2 \left(\frac{\lceil\log_c m^2\rceil}{\left(m - \left\lceil\sqrt{\lceil\log_c m^2\rceil}\right\rceil + 1\right)^2} + \frac{\alpha}{m^2} \right) \\ = \; & n^2 \frac{4\lceil\log_c m^2\rceil + \alpha}{m^2} \\ = \; & O\left(\frac{n^2}{m^2}\log_c m^2\right) = O\left(\frac{|T|}{|P|}\log_c |P|\right), \end{aligned}$$

where we have used the fact that $m - \left\lceil\sqrt{\lceil\log_c m^2\rceil}\right\rceil + 1 \geq m/2$ if $c \geq 2$ and $m \geq 2$.

We have shown the following result.

THEOREM 3.1. *The occurrences of an $m \times m$ pattern P in an $n \times n$ text T can be found using square templates of size $\lceil\log_c m^2\rceil$ in expected time $O(\frac{n^2}{m^2}\log_c m^2)$. The preprocessing of P takes time and space $O(m^2 \log_c m^2)$.* □

It should be clear that the algorithm of this section generalizes to d-dimensional pattern matching for rectangular patterns P and texts T such that the expected running time is $O\left(\frac{|T|}{|P|}\log_c |P|\right)$ for processing T. Preprocessing P needs time $O(|P| \log_c |P|)$.

4 Linear templates

In this section we relax the requirement of minimal number of text samples. Instead, we minimize the number of text entries belonging to samples by letting the samples share entries. This together with an efficient method for reading the text samples and comparing them to pattern samples leads to an algorithm with performance similar to the algorithm of previous section.

To make the reading of the text samples simple in the case of overlapping templates, we use *linear templates.* A linear template Q of size q defines the shape of samples to be an evenly spaced subsequence of q entries on a row of a two-dimensional array. Hence Q is of the form $Q = ((0,h),(0,2h),\ldots,(0,(q-1)h))$ for some $h \geq 1$. Note that samples $Q(i,j)$ and $Q(i,j+h)$ share $q-1$ entries.

Text sampling scheme $\mathcal{Q}_T$ will consist of linear samples placed on every mth row, called a *test row,* such that they start from every hth entry. Hence the samples overlap each other as $Q(i,j)$ and $Q(i,j+h)$ above. This gives

$$\mathcal{Q}_T = \{Q(im, jh) \mid 1 \leq i \leq \left\lfloor\frac{n}{m}\right\rfloor, \; 1 \leq j \leq \left\lfloor\frac{n}{h}\right\rfloor - q + 1\}.$$

The number of text entries belonging to samples in $\mathcal{Q}_T$ is now $\lfloor n/m\rfloor\lfloor n/h\rfloor$. To minimize this, we should maximize h. Upper bound for h is given by the fact that every potential occurrence must contain at least one text sample. This gives $h = \lfloor m/q\rfloor$. The pattern sampling scheme is

$$\mathcal{Q}_P = \{Q(i,j) \mid 1 \leq i \leq m, \; 1 \leq j \leq h\}.$$

Note that there are no overlaps between pattern samples.

Utilizing the overlaps of text samples makes it possible to read each string $s(T, Q(i,j))$, $Q(i,j) \in \mathcal{Q}_T$,

in constant time. For each test row, we read the sample entries from left to right and maintain a window of size q to hold the current string. We also need to find the matching pattern samples in constant time.

To achieve this we replace the trie in Section 3 with an Aho-Corasick multipattern matching automaton $AC(P, \mathcal{Q}_P)$ (see [1]). The automaton can be built in the same asymptotic time as the trie; in fact $AC(P, \mathcal{Q}_P)$ is simply the trie augmented with the *failure*-transitions. The addresses of the test strings are attached to the states of $AC(P, \mathcal{Q}_P)$ that correspond to the test strings. As there are $m\lfloor m/q \rfloor$ samples of size q in $\mathcal{Q}_P$, the preprocessing time becomes $O(m^2)$; with c shown explicitly the bound becomes $O(m^2 c)$ or $O(m^2 \log c)$. This is smaller, by a factor of q, than in the previous section as there is now fewer pattern samples.

The method of treating strings as integers can also be modified to handle the situation. During the scanning of T we treat the characters as digits and maintain a window of q digits. The preprocessing of P stays the same, except that the processing time drops to $O(m^2)$.

4.1 Analysis.

As already mentioned, the number of text entries read during elimination is

$$\left\lfloor \frac{n}{m} \right\rfloor \left\lfloor \frac{n}{\lfloor m/q \rfloor} \right\rfloor < \frac{n^2}{m\lfloor m/q \rfloor}$$

which is also the running time of elimination phase as each entry can be processed in constant time. The expected running time of the checking phase is, as before, at most $n^2 \frac{\alpha}{c^q}$. Thus, the total expected text processing time is now

$$n^2 \left(\frac{1}{m\lfloor \frac{m}{q} \rfloor} + \frac{\alpha}{c^q} \right).$$

The minimum is again formed by taking the derivative (the floor function omitted) which gives

$$q = \log_c m^2 + \log_c \frac{c \ln c}{c-1},$$

where $0 < \log_c \frac{c \ln c}{c-1} < \frac{1}{2}$ for $c \geq 2$. Thus we again choose $q = \lceil \log_c m^2 \rceil$ * which leads to

$$\begin{aligned} n^2 \left(\frac{2 \log_c m^2 + \alpha}{m^2} \right) &= O\left(\frac{n^2}{m^2} \log_c m^2 \right) \\ &= O\left(\frac{|T|}{|P|} \log_c |P| \right) \end{aligned}$$

for the total expected number of examined symbols during elimination and checking.

THEOREM 4.1. *The occurrences of an $m \times m$ pattern P in an $n \times n$ text T can be found using overlapping linear templates of size $\lceil \log_c m^2 \rceil$ in expected time $O(\frac{n^2}{m^2} \log_c m^2)$. The preprocessing of P takes time and space $O(m^2)$.* □

The algorithm again easily generalizes to d-dimensional P and T. Note that, for any d, the generalized algorithm uses linear (1-dimensional) samples scanned with the Aho-Corasick automaton. In this way the elimination phase of d-dimensional matching reduces directly to 1-dimensional techniques.

For larger d it may happen that $q = \lceil \log_c m^d \rceil > m$ in which case the straightforward generalization does not work any more.† An easy correction is to use superrows, consisting of two or more adjacent one-dimensional rows. As $2^{d-1} m \geq \log_c m^d$ for $d \geq 1$, $c \geq 2$, $m \geq 2$, a superrow with diameter 2 is always sufficient. Therefore this does not change the asymptotic behaviour of the algorithm.

The algorithm of this section is quite similar to the (suboptimal) algorithm by Baeza-Yates & Regnier [4], the main difference being that we use test samples with fixed size and location.

5 Lower bound

In [20] Yao proved a lower bound of $\Omega\left(\frac{n}{m} \log_c m\right)$ for the expected running time of one-dimensional pattern matching. In this section we present a generalization of Yao's result for d-dimensional hypercubic strings.

Let P be a pattern of size m^d and T a text of size n^d over alphabet Σ of size c. There exists some minimum number of entries of T that must be examined before any algorithm can be sure that all occurrences of P in T have been found. We define $g(P, n)$ to be the minimum of such numbers over all texts of size n^d. In a way, this is the best case lower bound for pattern P.

DEFINITION 5.1. *For $n \geq m > 0$, let*

$$f_1(m, n) = \left\lceil \log_c \left(\frac{(n-m)^d}{\ln(m^d + 1)} + 2 \right) \right\rceil$$

and

$$f_2(m, n) = \left\lfloor \frac{n}{2m} \right\rfloor^d \lceil \log_c (m^d + 1) \rceil.$$

Define

$$f(m, n) = \begin{cases} f_1(m, n) & \text{if } m \leq n \leq 2m, \\ f_2(m, n) & \text{if } n > 2m. \end{cases}$$

*Taking the floor function into account, $q' = \left\lfloor \frac{m}{\lfloor m/q \rfloor} \right\rfloor$ is always at least as good a choice as q, because $q' \geq q$ and $\lfloor m/q' \rfloor = \lfloor m/q \rfloor$.

†For $d = 2$ there is one such case, namely $m = 3$, $c = 2$ with $\log_2 3^2 \approx 3.17$. In this special case we can simply set $q = 3$.

THEOREM 5.1. *There exists a positive number a depending only on d such that, for any $|\Sigma| = c \geq 2$ and $m > 0$, there exists a set of patterns $L \subseteq \Sigma^{m^d}$ satisfying*

$$|L| \geq \left(1 - \frac{1}{m^{9d}}\right) c^{m^d},$$

and

for each $P \in L$, $g(P,n) \geq af(m,n)$ for all $n \geq m$.

Proof. The proof is a simple modification of Yao's proof in [20]. □

Above theorem shows that for majority of patterns even the best case complexity of pattern matching is

$$\Omega\left(\frac{n^d}{m^d} \log_c m^d\right),$$

for a fixed d. This implies that for a fixed d and T and varying P, the expected running time of pattern matching is $\Omega\left(\frac{n^d}{m^d} \log_c m^d\right)$.

6 The k mismatches case

The *k mismatches problem* asks for finding approximate occurrences of P with at most k mismatches, that is, occurrences that match with P except possibly for $\leq k$ symbols. In this section we generalize the algorithms of Section 3 and Section 4 to the k mismatches problem.

One of the requirements of the algorithms of previous sections was that every potential occurrence has exactly one witness. The k mismatches case, $k \leq 1$, can be handled by increasing the number of witnesses per potential occurrence as shown by the following lemma.

LEMMA 6.1. *Let potential occurrence R have $k+p$ disjoint witnesses. If $< p$ of them are positive then R can not be an occurrence of P with $\leq k$ mismatches.* □

Suggested by the lemma, the sampling scheme $\mathcal{Q}$ has to be such that every potential occurrence has $k+p$ non-overlapping witnesses of size q. Parameters q and p will be determined in the analysis; note that to get optimal performance we can not simply choose $p = 1$.

The elimination phase now maintains a counter, initially equal to 0, for every potential occurrence R. When a positive witness of R is found the counter of R is increased by one. When the counter achieves value p, R is transmitted to the checking phase. The checking phase naively compares R and P until $k+1$ mismatches are found or the whole pattern has been compared.

We will now show how to modify the sampling schemes of Sections 3 and 4 to have $k+p$ witnesses for each potential occurrence.

6.1 Square templates.

The text sampling scheme described in Section 3 consists of square samples placed in the form of a regular grid such that every potential occurrence contains exactly one whole sample. If we take $k+p$ copies of such sample grid, each translated such that there are no overlaps, we have a text sampling scheme $\mathcal{Q}_T$ where each potential occurrence contains exactly $k+p$ disjoint text samples. When we define the pattern sampling scheme $\mathcal{Q}_P$ to be the same as in Section 3, we have the desired sampling scheme $\mathcal{Q} = (\mathcal{Q}_P, \mathcal{Q}_T)$ with $k+p$ witnesses per potential occurrence.

The method works as long as we can keep the text samples disjoint. This means that we must have

$$k+p \leq \left\lfloor \frac{m - \lceil\sqrt{q}\rceil + 1}{\lceil\sqrt{q}\rceil} \right\rfloor^2.$$

6.2 Linear templates.

Using the linear templates of Section 4, there are two ways to increase the number of witnesses. First, we can increase the number of test rows in $\mathcal{Q}_T$ so that every potential occurrence contains u test rows. If $k+p \leq m$, we can select $u = k+p$ and we are done. In the case $k+p > m$, every segment of length m of every test row in $\mathcal{Q}_T$ will contain more than one, say v, disjoint samples. This is achieved by selecting a step size $h = \lfloor m/vq \rfloor$. The text sampling scheme can now be defined as

$$\begin{aligned}\mathcal{Q}_T = \Big\{ Q(im-u+r, jh) \;\Big|\; & 1 \leq j \leq \left\lfloor \frac{n}{h} \right\rfloor - q + 1 \\ & 1 \leq i \leq \left\lfloor \frac{n}{m} \right\rfloor,\ 1 \leq r \leq \min\{u, n - im + u\} \Big\}.\end{aligned}$$

The corresponding pattern sampling scheme is

$$\begin{aligned}\mathcal{Q}_P = \{ Q(i, j+rqh) \;\mid\; & 0 \leq r \leq v-1, \\ & 1 \leq i \leq m,\ 1 \leq j \leq h \}.\end{aligned}$$

This sampling scheme $\mathcal{Q} = (\mathcal{Q}_P, \mathcal{Q}_T)$ gives uv disjoint witnesses per potential occurrence.

For some (large) values of q and $k+p$ it may not be possible to select u and v such that the optimum value $k+p$ of uv is achieved. However, by selecting

$$v = \left\lceil \frac{k+p}{m} \right\rceil \text{ and } u = \left\lceil \frac{k+p}{v} \right\rceil,$$

we can get close as then

$$\begin{aligned} k+p \leq uv &= \left\lceil \frac{k+p}{v} \right\rceil v \leq k+p+v-1 \\ &= k+p+\left\lceil \frac{k+p}{m} \right\rceil - 1 \\ &< (k+p)\left(1 + \frac{1}{m}\right). \end{aligned}$$

As v and u always must satisfy $vq \leq m$ and $u \leq m$, the above selections can be done as long as

$$k + p \leq m \left\lfloor \frac{m}{q} \right\rfloor. \tag{6.1}$$

6.3 Analysis

(Sketch). The running time (without pattern preprocessing) is the sum of three components:

A = the number of entries of T examined during the elimination,

B = the number of updates of the counters for potential occurrences R,

C = the number of symbols compared during the checking.

For both of the above sampling schemes we have

$$A \approx \frac{n^2(k+p)q}{m^2}. \tag{6.2}$$

The expected value $\bar{B}$ of B is

$$\bar{B} \leq \frac{n^2(k+p)}{c^q} \tag{6.3}$$

because there are at most n^2 potential occurrences with $k + p$ witnesses each, and a witness is positive with probability $1/c^q$.

The third component C needs a more careful analysis. First, we denote by C' the number of character comparisons during naive checking of a fixed R.* As C' is a Pascal distributed random variable with parameters $k + 1$ and $\frac{c-1}{c}$, we get

$$\bar{C}' = (k+1)\frac{c}{c-1} \leq 2(k+1).$$

Next we analyse the probability, denoted by H, that the checking is started for a fixed R. As H is the probability that at most k of the $k + p$ witnesses of R are negative, we have

$$H = \sum_{i=0}^{k} \binom{k+p}{i} \left(\frac{1}{c^q}\right)^{k+p-i} \left(1 - \frac{1}{c^q}\right)^i.$$

Noting that

$$\binom{k+p}{i} = \frac{\binom{k+p}{k}\binom{k}{i}}{\binom{k+p-i}{k-i}},$$

we can rewrite H as follows

$$\begin{aligned} H &= \frac{\binom{k+p}{k}}{c^{qp}} \sum_{i=0}^{k} \frac{\binom{k}{i}}{\binom{k+p-i}{k-i}} \left(\frac{1}{c^q}\right)^{k-i} \left(1 - \frac{1}{c^q}\right)^i \\ &\leq \frac{\binom{k+p}{k}}{c^{qp}} \sum_{i=0}^{k} \binom{k}{i} \left(\frac{1}{c^q}\right)^{k-i} \left(1 - \frac{1}{c^q}\right)^i \\ &= \frac{\binom{k+p}{k}}{c^{qp}}. \end{aligned}$$

As there are at most n^2 potential occurrences, we have

$$\bar{C} \leq n^2 H \bar{C}' \leq \frac{2n^2(k+1)\binom{k+p}{k}}{c^{qp}}. \tag{6.4}$$

Select finally

$$q = \lceil log_c m^2 \rceil \text{ and} \tag{6.5}$$

$$p = 1 + \left\lceil \log_{c^q} \binom{k+p}{k} \right\rceil. \tag{6.6}$$

Value p satisfying this implicit definition can, in practice, be found by trying with $p = 1, 2, \ldots$ until the right (typically very small) value is found. If this gives $p > k$, we define $p = k$. This can happen only if $k \leq 1$ or $m \leq 2$, and even then it would hold that $p \leq k + 1$. Substituting (6.5) and (6.6) into (6.2), (6.3), and (6.4) gives

$$\begin{aligned} A &= O\left(\frac{n^2(k+1)\log_c m^2}{m^2}\right) \text{ and} \\ \bar{B}, \bar{C} &= O\left(\frac{n^2(k+1)}{m^2}\right). \end{aligned}$$

Hence the expected running time of our algorithm is

$$A + \bar{B} + \bar{C} = O\left(\frac{n^2(k+1)\log_c m^2}{m^2}\right).$$

The algorithm works only if $(k+p)q \leq m^2$ because each potential occurrence has to have $k + p$ disjoint witnesses. Moreover, $k + p$ has to satisfy (6.1) in order to make correct sampling scheme possible. The above analysis holds true for $q = \lceil \log_c m^2 \rceil$ and for some $1 \leq p \leq k$. These conditions altogether imply that the algorithm certainly works and the above analysis is valid at least if $k \leq \frac{m^2}{4\lceil \log_c m^2 \rceil}$, but in some cases also higher values of k up to $\frac{m^2}{\lceil \log_c m^2 \rceil} - p$ can be possible.

The next theorem follows.

THEOREM 6.1. *Let $k \leq \frac{m^2}{4\lceil \log_c m^2 \rceil}$. Then the k mismatches problem for two-dimensional $m \times m$ pattern P and $n \times n$ text T can be solved in expected time $O(\frac{n^2(k+1)\log_c m^2}{m^2})$. The preprocessing of P takes time and space $O(m^2)$.* □

*We ignore here the fact that such an R has passed the elimination phase and thus has less mismatches on average, which might increase the checking time. On the other hand, if we know the position of positive witnesses, we could use this to speed up checking.

The algorithm requires that $k = O(m^2/\log_c m^2)$. Substituting this into the bound of the above theorem shows that whenever the algorithm works, it is at most linear in $|T|$. Chang and Lawler [8] have obtained similar results in one-dimensional case; their algorithm also works for the k differences problem.

7 Concluding remarks

There are many possibilities for practical fine-tuning of our algorithms. For example, in the algorithm of Section 3 it is possible to combine the elimination and checking phases using spiral-shaped test strings of Gonnet [12]. The text processing in all algorithms can be performed on-line, with a relatively small window into the text. In the algorithms for exact matching a window of size $O(m^2)$ suffices; the algorithm for k mismatches needs window size $O(nm)$.

We ignored worst-case considerations in this paper. It is possible to use some linear worst-case algorithm for the checking phase of our algorithms such that the total time stays $O(n^2)$. However, it is not clear that this would be useful in practice. More important is that the obliviousness of the elimination phase makes parallel implementations of our algorithms simple.

References

[1] A.V. Aho and M.J. Corasick: Efficient string matching: An aid to bibliographic search. *Comm. ACM* **18** (1975), 333–340.

[2] A. Amir and G. Benson: Two-dimensional periodicity and its applications. *Proc. 3rd ACM-SIAM SODA*, 1992, 440–452.

[3] A. Amir, G. Benson and M. Farach: Alphabet independent two dimensional matching. *Proc. 24th ACM STOC*, 1992, 59–68.

[4] R. Baeza-Yates and M. Regnier: Fast two-dimensional pattern matching. *Inform. Process. Lett.* **45** (1993), 51–57.

[5] T.J. Baker: A technique for extending rapid exact-match string matching to arrays of more than one dimension. *SIAM J. on Comput.* **7** (1978), 533–541.

[6] R.S. Bird: Two dimensional pattern matching. *Inform. Process. Lett.* **6** (1977), 168–170.

[7] R. Boyer and S. Moore: A fast string searching algorithm. *Comm. ACM* **20** (1977), 762–772.

[8] W. Chang and E. Lawler: Approximate string matching in sublinear expected time. *Proc. 31st IEEE FOCS*, 1990, 116–124.

[9] M. Crochemore, L. Gasieniec and W. Rytter: Two-dimensional pattern matching by sampling. *Inform. Process. Lett.* **46** (1993), 159–162.

[10] M. Crochemore, W. Rytter: Efficient Algorithms on Texts. Manuscript of a monograph.

[11] Z. Galil and K. Park: Truly alphabet-independent two-dimensional pattern matching. *Proc. 33th IEEE FOCS*, 1992, 247–256.

[12] G.H. Gonnet: Efficient searching of text and pictures (extended abstract). Tech. Rep. OED-88-02, Centre for the new OED, University of Waterloo, 1988.

[13] A. Hume and D. Sunday: Fast string searching. *Software — Practice and Experience* **21** (1991), 1221–1248.

[14] R. Karp and M. Rabin: Efficient randomized pattern-matching algorithms. *IBM J. Res. Develop.* **31** (1987), 249–260.

[15] J.Y. Kim and J. Shawe-Taylor: Fast expected two dimensional pattern matching. *Proc. First South American Workshop on String Processing* (ed. R. Baeza-Yates and N. Ziviani), 1993, 77–92.

[16] D.E. Knuth, J.H. Morris and V.B. Pratt: Fast pattern matching in strings. *SIAM J. on Comput.* **6** (1977), 323–350.

[17] S. Ranka and T. Heywood: Two-dimensional pattern matching with k mismatches. *Pattern Recognition* **24** (1991), 31–40.

[18] J. Tarhio: A Boyer-Moore approach for two-dimensional matching. Technical Report, Division of Computer Science, University of California at Berkeley, 1993.

[19] E. Ukkonen: Approximate string-matching with q-grams and maximal matches. *Theoretical Computer Science* **92** (1992), 191–211.

[20] A.C. Yao: The complexity of pattern matching for a random string. *SIAM J. on Comput.* **8** (1979), 368–387.

[21] R.F. Zhu and T. Takaoka: A technique for two-dimensional pattern matching. *Comm. ACM* **32** (1989), 1110–1120.